Universidad Tecnológica Nacional
Facultad Regional Córdoba
Ing. en Sistemas de Información

EXAMEN PARCIAL 4

Algoritmos y Estructuras de Datos

Fecha: 18/10/2025

Turno T3 - Enunciado:

Una organización de eSports necesita un sistema para gestionar los equipos que participarán en el Torneo Nacional de Valorant. Cada equipo se registra con sus datos y resultados obtenidos en competencias anteriores, con el fin de analizar su desempeño. De cada *Equipo* se conoce: nombre (cadena de caracteres), cantidad de jugadores (entero positivo, incluye titulares y suplentes), rango promedio (entero entre 1 y 8, donde: 1–Hierro, 2–Bronce, 3–Plata, 4–Oro, 5–Platino, 6–Diamante, 7–Inmortal, 8–Radiante), puntaje total en torneos (entero positivo), y región competitiva (entero entre 1 y 4: 1–LATAM Norte, 2–LATAM Sur, 3–Norteamérica, 4–Europa).

En base a lo anterior, se provee **ya desarrollado**:

- ✓ Un módulo *clase.py* que contiene la definición de la clase **Equipo** con el *método constructor* y el método *__str__()* ya incluidos. NO CAMBIE NADA. NO AGREGUE NADA. DEJE LA CLASE TAL CUAL ESTÁ.
- ✓ Un segundo módulo *principal.py* que contiene una función *main()* con el esquema del menú de opciones ya desarrollado y funcionando, de forma que esa función *main()* sea usada como la función de entrada del programa completo, y una función de validación de carga. En el mismo módulo, se provee ya desarrollada una función *generar_aleatorio()* que crea con valores aleatorios un objeto de la clase **Equipo** y lo retorna ya listo para usar. Los procesos para cumplir con cada opción del menú, son justamente la tarea que debe programar cada estudiante. Las opciones a programar son:

---

1. Cargar un arreglo de registros con los datos de *n* equipos (cargar *n* por teclado y validar que sea positivo), de manera que en todo momento el arreglo se mantenga ordenado por **nombre** del equipo. Para esto debe utilizar el algoritmo de inserción ordenada con búsqueda binaria (se considerará directamente incorrecta la solución basada en cargar el arreglo completo y ordenarlo al final, o aplicar el algoritmo de inserción ordenada, pero con búsqueda secuencial). La carga de los equipos debe ser automática y con base aleatoria (use la función *generar_aleatorio()* que ya tiene disponible en el módulo *principal.py* para crear los objetos). Cada vez que se ingrese en esta opción, el arreglo debe ser creado desde cero y todo contenido anterior debe ser eliminado.

   Ajuste de runner: al ejecutar esta opción, solo cargue por teclado el valor de n, y no muestre nada al terminar de cargar el vector. Solo regrese al menú.

---

2. Mostrar todos los equipos del arreglo, a razón de un registro por línea, y contar y mostrar cuántos pertenecen al rango promedio *r1* (r1 es ingresado por teclado).

   Ajuste de runner: al ejecutar esta opción, cargue por teclado el valor de r1, muestre el listado pedido usando el método __str__() que ya tiene en la clase, muestre al final el promedio pedido aquí (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

---

3. Obtener la cantidad de equipos según el rango promedio y la región competitiva (O sea: 8 x 4 contadores en una matriz de conteo). Mostrar únicamente los contadores que sean mayores a cero, indicando claramente el rango y la región correspondientes.

   Ajuste de runner: al ejecutar esta opción, asegúrese de que la matriz sea de orden 8 x 4 (Y NO 4 x 8). Para mostrarla, recórrala por filas, muestre cada contador pedido indicando a qué rango y a qué región corresponde cada uno (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

---

4. A partir del arreglo, generar un archivo binario donde se incluyan todos los datos de los equipos cuyo puntaje total sea mayor o igual a 600 puntos (o sea, llegar a rango Premier, que son los aptos para participar en torneos de nivel semi-profesional).

   Ajuste de runner: al ejecutar esta opción, solo genere el archivo pedido y regrese al menú (no muestre nada en la pantalla al terminar).

5. Mostrar el contenido del archivo binario, a razón de un registro por línea en pantalla, indicando además una línea extra que muestre el promedio de puntaje total de los equipos pertenecientes al rango promedio *Diamante.*

   Ajuste de runner: al ejecutar esta opción, muestre el listado pedido usando el método __str__() que tiene disponible en la clase, muestre al final el promedio pedido aquí (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

Criterios generales de evaluación.

- Estructura completa del programa terminado **[máximo: 4 puntos por convenciones de estilo, la estructura de las funciones que desarrolle, el control de ejecución del módulo principal, todo otro elemento que aporte a la estructura del programa]**
- Desarrollo correcto del ítem 1: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 2: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 3: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 4: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 5: **[máximo: 4 puntos]**
- Para aprobar el parcial (con nota mínima de 4(cuatro), el alumno debe llegar a un *porcentaje de al menos 55% del puntaje máximo (eso es, un poco más de 13 puntos)*.